

ActNAS : Generating Efficient YOLO Models using Activation NAS

Sudhakar Sah, Ravish Kumar, Darshan C. Ganji, Ehsan Saboori

Deeplite, Inc.
Toronto, Canada
sudhakar@deeplite.ai

Abstract

Activation functions introduce non-linearity into neural networks, allowing them to learn complex patterns. Different activation functions impact differently on speed and accuracy; for instance, ReLU is fast but often less precise, while SiLU offers higher accuracy at the expense of speed. Traditionally, a single activation function is used throughout a model. In this work, we conducted a comprehensive study on the effects of using mixed activation functions in YOLO-based models, examining their impact on latency, memory usage, and accuracy across CPU, NPU, and GPU edge devices. We propose Activation NAS (ActNAS)—a Hardware-Aware Neural Architecture Search (HA-NAS) method that optimizes activation functions per layer for specific hardware. ActNAS-generated models maintain comparable mean Average Precision (mAP) to baselines, while achieving up to $1.67\times$ faster inference and/or 64.15% lower memory usage. Additionally, we demonstrate that hardware-aware models learn to leverage architectural and compiler-level optimizations, resulting in highly efficient performance tailored to each hardware platform.

1. Introduction

Improving the accuracy of computer vision models is a highly competitive area, with ongoing efforts to refine or develop new architectures that achieve state-of-the-art performance. For example, the recent YOLO10 [25] has set a new benchmark for object detection in the YOLO family [23], outperforming earlier models on the Microsoft COCO dataset [12]. The push for higher accuracy often involves designing more complex models—either by increasing learnable parameters or introducing performance-enhancing blocks—which typically results in higher latency.

To address this, some works have focused on redesigning activation functions to improve the trade-off between accuracy and latency. As these functions are applied throughout

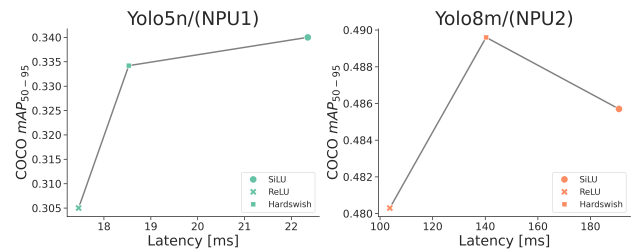


Figure 1. Latency-mAP plot of YOLO5n and YOLO8m for two different NPUs. SiLU models are slowest and ReLU models are fastest on both devices

the model, they have a significant impact on both. For instance, SiLU [4] yields better accuracy but higher latency compared to ReLU [2], while HardSwish [7] provides the best balance between the two. Figure 1 illustrates this trade-off for YOLO5n and YOLO8m models.

Despite their influence, it is common to use a single activation function across all layers. Through our exploration, we found that using different activation functions per layer can either slightly improve mAP (by less than 1%) or keep it within a 1% drop while reducing latency by 30–70%, compared to models using SiLU exclusively. In this study, we controlled all other variables affecting latency and mAP to isolate the effect of mixed activation usage.

To automate this process, we propose a Hardware-Aware Neural Architecture Search (HA-NAS) method to select the optimal activation function for each layer based on specific latency and accuracy constraints.

We summarize our contributions as follows:

- Introducing ActNAS, which searches for optimal mixed activations per layer
- Quantifying memory and latency impacts of different activations across edge devices.
- Demonstrating significant latency gains with minimal mAP loss using ActNAS models on the COCO dataset.
- Showing that models optimized for one hardware plat-

form may not generalize well to others.

2. Related Works

In building computer vision models, it is a common practice to use the same activation function across all layers of the architecture. For example, YOLO models [23], whether in the n, m, s, l, or tiny variants, consistently use the SiLU [4] activation function due to its superior accuracy compared to alternatives like ReLU [21]. However, this higher accuracy often comes at the cost of increased latency, which opens up the possibility of exploring other activation functions such as ReLU [2], LeakyReLU[26], or Hardswish [7]. While YOLO models do allow the option to switch from SiLU to other activations across the entire architecture, there has been no prior research (based on our knowledge) on using mixed activation functions within the same YOLO model. The closest existing work involves designing hybrid activation functions, simplifying blocks in the architecture by applying a different activation function than the main one or mixing activations in image classification models.

Hybrid Activation: In [20], the authors combined softmax [19] and sparsemax[14] in the final activation layer of a Convolutional Neural Network (CNN) for gait analysis using silhouettes. Similarly, in [3], the authors proposed an activation function that can replace ReLU, SiLU, and Hardswish in deep learning models, providing a hybrid approach.

Block Simplification: YOLOv6-3.0 [10] introduced the idea of simplifying the neck of YOLO models [23] by replacing the SiLU activation in the SPPF block [6] with ReLU, creating the SimSPPF block. This was further modified into the SimCSPSPF block to enhance performance. These works demonstrate that different activation functions have unique strengths and weaknesses, motivating us to explore mixed activations within a single YOLO architecture.

Designing Model Architecture using NAS: In [28], Zoph demonstrated how to design Recurrent Neural Networks (RNNs) with constituent nodes using different activation functions such as ReLU, identity, tanh, or sigmoid. Building on this, [17] introduced a method to search across activation functions like ReLU and Gaussian-smoothed ReLU. Zhenyu [27] further proposed Eigen-NAS, a train-free algorithm that searches for optimal skip connections and activations for each layer in image classification models, showing improvements on datasets like CIFAR-100 and ImageNet-16 but observed a drop in accuracy with CIFAR-10. Our work extends this approach of mixing activations to YOLO architectures by proposing ActNAS, a solution that optimizes activation functions within YOLO models based on layer-specific accuracy and latency data.

Zero-Cost (ZC) Estimators: Zero-cost estimators, which provide accuracy predictions without full training, have been shown to significantly reduce training time. In

[9], Ivan et al. analyzed various ZC estimators, including those from [1, 11, 15], in the context of YOLO models, concluding that the NWOT metric [15] performs well for networks using different activations. However, ZC estimators have not yet been applied to assess the micro-level impact of architectural changes, such as altering individual activation functions within a model.

3. Methodology

Mixed activation models aim to identify the optimal combination of activation functions for each layer, enabling a more effective trade-off between latency and accuracy. As shown in Figure 1, YOLO5n and YOLO8m models using SiLU activation achieve higher mAP but are the least efficient across different devices. In contrast, models using ReLU activation are the fastest but have lower mAP. By incorporating multiple activation functions within the same architecture, mixed activation models go beyond the conventional single-function design and offer a pathway to tailor performance based on hardware constraints and application needs.

For our experiments, we used YOLO5n combined with YOLO8 head (which offers better accuracy) and YOLO8m model, with the SiLU-based model serving as a reference. As shown in Figure 4, we generate a search space by systematically replacing each activation function in the reference model with a set of candidate activations: [*ReLU*, *SiLU*, *Hardswish*, *ReLU6*, and *LeakyReLU*]. We replace one activation at a time, as depicted in Figure 2, resulting in 345 candidate models for YOLO5n (since the reference model has 69 activations and there are 5 candidate activations for each layer).

To evaluate the impact of each activation replacement, we create a new model for each candidate and measure its accuracy, latency, and memory usage, comparing it to the reference model. This process is iterative and is repeated for all 345 candidate models. The results are recorded in performance tables: the latency table, accuracy table, and memory table. Each entry in the accuracy table includes the layer name, activation name, reference accuracy, and delta accuracy (the difference between the reference model’s accuracy and the candidate model). The latency and memory tables follow the same process and record latency and memory values instead of accuracy.

3.1. Accuracy Estimator

As the number of activations and layer combinations increase, the search space grows exponentially, making it computationally expensive and time-consuming to train each model and evaluate the impact of replacing activations. To address this, we use the NWOT Zero-Cost (ZC) metric [15] to approximate the impact of changing activation functions in different layers on the accuracy of the model.

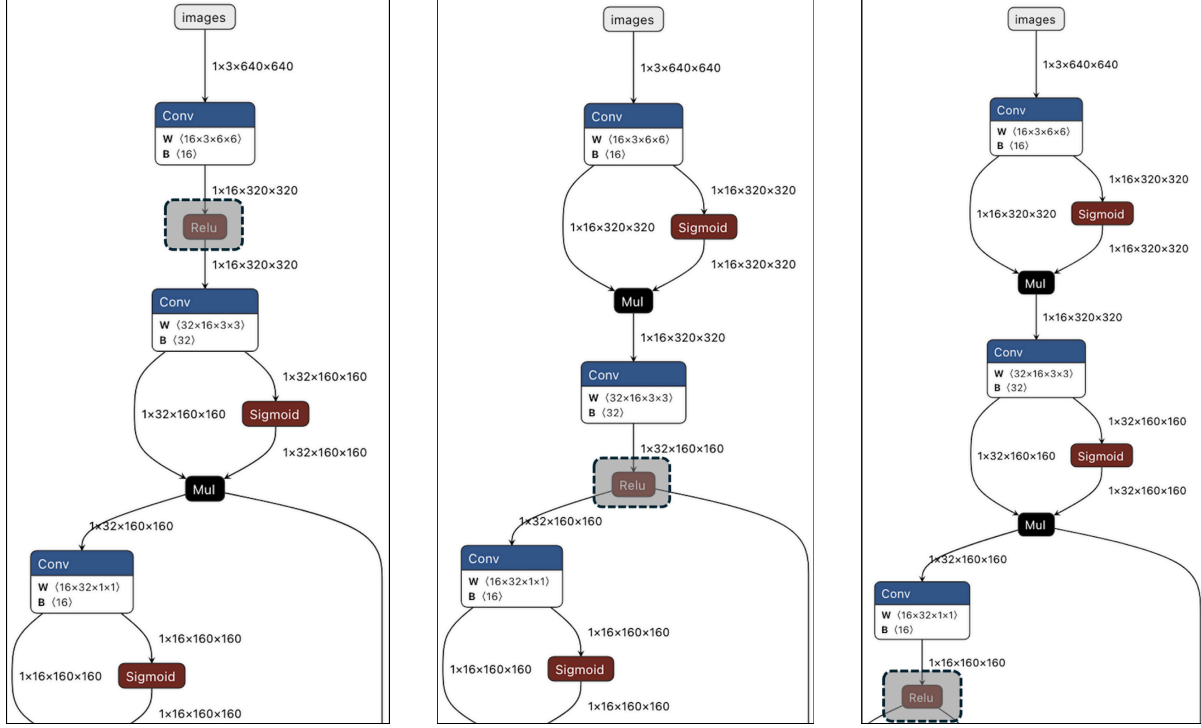


Figure 2. Layer wise activation replacement a) Replace first SiLU activation to ReLU b) Replace second activation c) Replace third activation

To validate the effectiveness of NWOT score as a proxy to the mAP of trained YOLO models, we estimated the correlation between the mAP of 131 variations of YOLO models in MCUBench [22] and their corresponding NWOT scores. Figure 3 shows that the NWOT score has a strong correlation with the mAP of fully trained models. The high correlation indicates that the NWOT score can reliably predict the impact of per-layer activation changes on accuracy without the need of training each candidate model. For each candidate activation replacement, the NWOT score is calculated, and the accuracy table is updated accordingly as described in the previous section. This approach significantly reduces the computational cost and time required to explore the large search space of mixed activation models. The suitability of zero cost proxy based method also opens an avenues for more complex NAS methods requiring larger search space.

3.2. Latency Computation

The impact of different activation functions on latency and memory depends on the hardware architecture, compiler level optimization and the runtime. For this reason, a variety of hardware, including two CPUs (ARM Cortex-A57 on Jetson Nano and ARM Cortex-A53 on Raspberry Pi 3), an embedded GPU (Jetson Nano), and three different reference neural processing units (NPU1, NPU2, NPU3) are used for

our experiments. We are not disclosing the vendor name and actual name of these NPUs as some of these are yet to be available in the market. To measure the performance of the models on these devices, we used ONNX Runtime [18] for Jetson Nano, TensorFlow Lite (TFLite) [13] for ARM Cortex processors, and custom compilers/runtimes as needed by each NPU. Each model is converted to ONNX, TFLite, or a custom format depending on the compiler. Latency values are recorded as average over 50 runs to ensure correctness. The input size of 224×224 or 640×640 is used depending upon the memory limitations. This variation in hardware, runtime and model family ensures that the results aren't biased towards a specific platform, runtime or model family, providing a balanced perspective of how different activations perform across different systems.

3.3. Mixed Activation Model Search

Latency, memory and accuracy table obtained from the benchmarking step is used to construct mixed activation models.

3.3.1. Local Zero Cost Maxima Approach

We started with the simple approach to create a model using just two activation functions, such as SiLU and ReLU. Starting with a reference model that uses only SiLU, we apply the Local Zero Cost Maxima (LZCM) method to it-

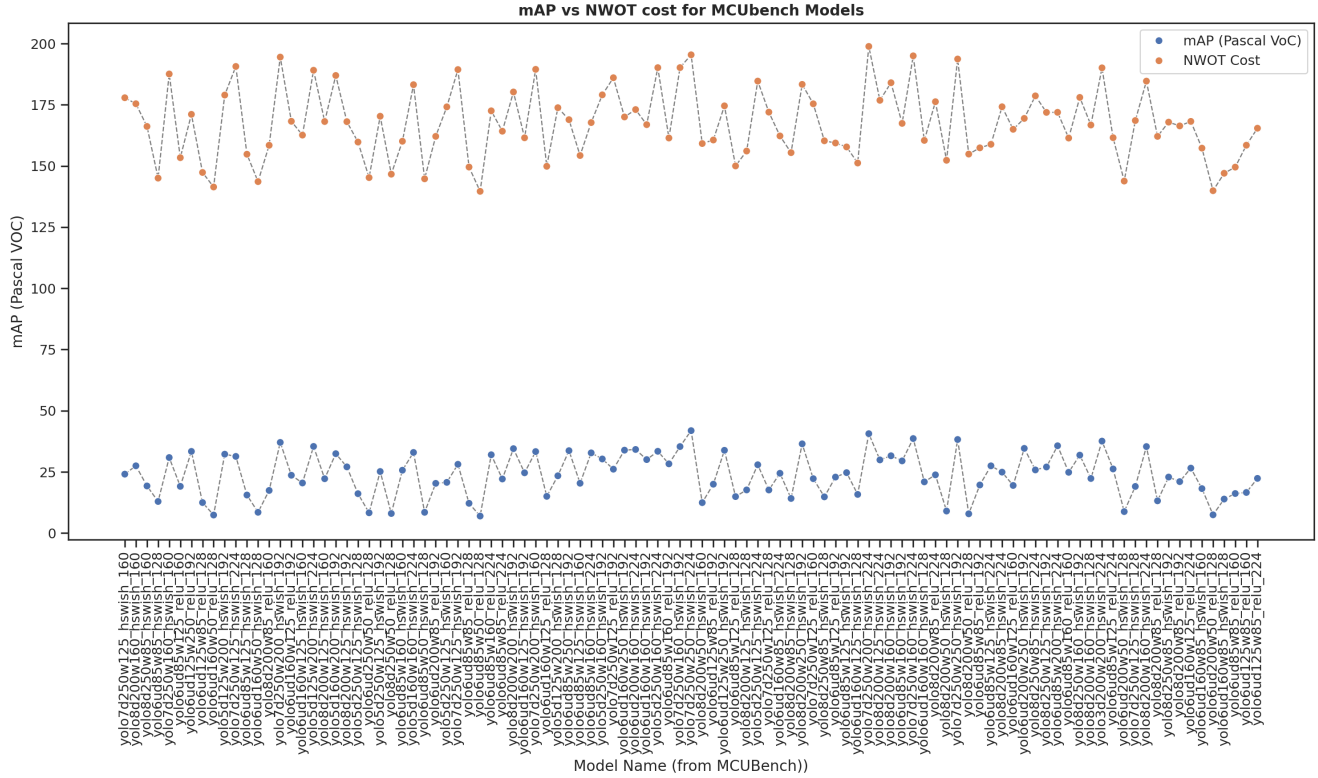


Figure 3. mAP vs NWOT cost for 131 MCUBench[22] Models. Each point represents one model and NWOT and mAP score of each model is plotted in orange and blue colors respectively

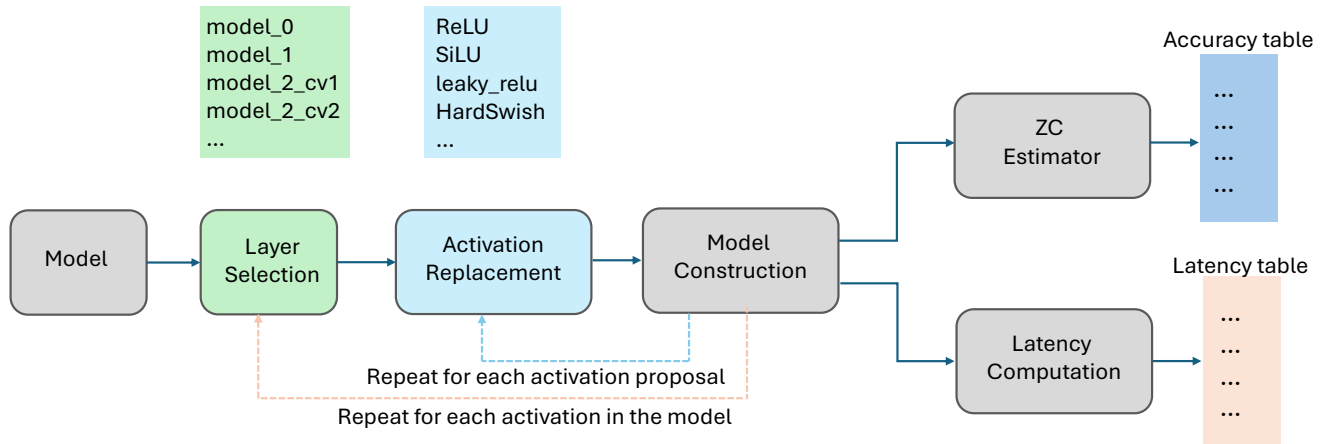


Figure 4. Model Benchmarking using training free estimators and on device inference. Each candidate model is created by replacing just one activation function.

eratively replace activation of a layer with ReLU provided the NWOT score is higher compared to the model that uses SiLU for the same layer. The modified model is then trained from scratch on the COCO dataset. Using this simple approach, we developed two models: LZCM1, by using SiLU and ReLU, and LZCM2, by using Hardswish and ReLU activations.

3.3.2. Naive Approach

Benchmarking models generated using the LZCM approach show that activations in layers closer to input have a significant impact on both latency and memory due to the larger feature sizes in those layers. Replacement of activations in initial layers with ReLU significantly reduces latency and

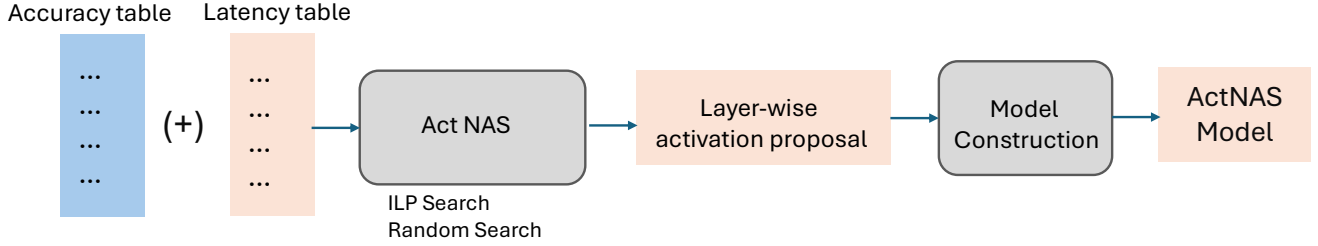


Figure 5. Activation NAS Process

memory usage. In our simple approach, we replace the first three activations near the input with ReLU, while keeping the remaining layers as SiLU. This leads to an efficient balance between performance and resource consumption.

3.3.3. Activation NAS Approach

The LZCM method follows a straightforward decision-making process, replacing activations in individual layers according to their zero-cost score. However, this approach only optimizes each layer locally and does not account for the combined impact of replacing activations in multiple layers at once. In contrast, the Activation NAS (ActNAS) approach searches for the optimal activation function for each layer while considering a global budget for latency, memory, or accuracy. The search space for ActNAS is defined by the number of layers multiplied by the number of activation function options. To navigate this space, we used the latency, memory, and accuracy tables, along with predefined constraints, to guide the search. We experimented with both random search and Integer Linear Programming (ILP) [24] to find the best activation configurations.

Random Search As shown in Figure 5, generating model proposals using Activation NAS requires precomputed latency and accuracy tables. These tables capture the changes in these metrics when a single layer’s activation in the original YOLO model (with SiLU activation) is replaced by an alternative activation. The tables are then converted into latency and accuracy cost matrices, where the rows represent the layers being modified, and the columns represent candidate activations (SiLU, ReLU, Hardswish, LeakyReLU and ReLU6). In addition to these matrices, we also define latency or accuracy constraints, which serve as the overall budget that the algorithm can work within when constructing new models with mixed activations. The random search process begins by randomly selecting activations from the search space that fit within the given constraints. Then it computes the total cost (in terms of latency or accuracy) for the model and marks it as the best possible model at that point. The algorithm repeats the random search, comparing each newly constructed model with the previous best.

If a new model has a better overall cost, it updates the best model definition and continues iterating through the random selection process.

ILP Search To generate model proposals with mixed activations, we first created latency and accuracy matrices, where each element represents the difference in metrics from the original SiLU-based YOLO model when a single-layer activation is replaced. We then formulated this problem as an Integer Linear Programming (ILP) [24] optimization problem. The objective was to minimize the cost of latency and accuracy while adhering to the constraints on the overall latency or accuracy of the newly constructed models. In the ILP formulation, one variable vector represents the indices of the layers where activations are applied, and the other represents the candidate activations for those layers. We used the open-source PuLP library [16], a Python toolkit for linear programming, to solve this ILP problem. The result is a ranking list of the top k model proposals. To ensure diversity among proposals, the solution avoids excessive overlap, keeping the model architectures meaningfully different.

The ActNAS models (ActNAS5n.1 is NPU1 aware and ActNAS5n.2 is Jetson Nano aware) in Table 1 and 2 are hardware aware mixed activation models (YOLO5n) which means that we used the latency of each candidate models on NPU1 to create the optimized models. Similarly, ActNAS8m.1 and ActNAS8m.2 models in Table 3 are NPU2 aware mixed activation models (YOLO8m). The ILP search can be adapted for different hardware targets by imposing a latency constraint on the constructed model.

3.4. Model Fine-tuning

We used Ultralytics [8] to train and fine-tune all our models. For consistency across experiments, we kept all hyperparameters exactly the same, training each model from scratch for 300 epochs using 640×640 COCO images and evaluating them on the COCO validation set which contains 5000 images. While mixed activation models could be fine-tuned for a few epochs starting from pre-trained weights, we chose to train all models from scratch to ensure a fair

Table 1. Performance of ActNAS models compared to baseline YOLO5n models on GPU and CPU

Model	mAP	Jetson Nano			Cortex A-53		
		Latency	Improvement (%)		Latency	Improvement (%)	
		(ms)	Hswish	SiLU	(ms)	Hswish	SiLU
SiLU	33.60	27.20	-	-	937.89	-	-
ReLU	31.88	20.18	-	-	793.27	-	-
Hardswish	33.13	25.63	-	-	823.91	-	-
LZCM1(SiLU/ReLU)	33.13	26.61	-21.54%	2.17%	916.97	-0.58%	2.23%
LZCM2(SiLU/Hswish)	33.29	31.15	-21.54%	-14.52%	828.72	-0.58%	11.64%
Naive(ReLU/SiLU)	33.52	24.55	4.21%	9.74%	906.78	-10.06%	3.32%
ActNAS5n.1	32.80	24.78	3.32%	8.90%	809.96	1.69%	13.64%
ActNAS5n.2	32.20	23.50	8.31%	13.60%	966.74	-17.34%	-3.08%

Table 2. Performance of ActNAS models compared to baseline models on NPU

Model	mAP	NPU1			NPU1		
		Latency	Improvement (%)		RAM	Improvement (%)	
		(ms)	Hswish	SiLU	(KB)	Hswish	SiLU
SiLU	0.3366	22.35	-	-	1230.00	-	-
ReLU	0.3188	17.46	-	-	588.00	-	-
Hardswish	0.3313	18.53	-	-	392.00	-	-
LZCM1(SiLU/ReLU)	0.3313	21.87	-5.50%	2.15%	1200.00	-206.12%	2.44%
LZCM2(SiLU/Hswish)	0.3329	19.55	-5.50%	12.53%	624.75	-59.38%	49.21%
Naive(SiLU/ReLU)	0.3352	21.43	-15.65%	4.12%	661.50	-68.75%	46.22%
ActNAS5n.1	0.3280	17.37	6.26%	22.28%	514.50	-31.25%	58.17%
ActNAS5n.2	0.3250	17.50	5.56%	21.70%	441.00	-12.50%	64.15%

comparison.

Table 3. Hardware aware performance of YOLO8m ActNAS models on different NPUs. All latency numbers are for 640×640 images. NPU1 is a low end NPU.

Model	mAP	Latency(ms)		
		NPU1 x100	NPU2	NPU3
SiLU	48.57	241.50	190.87	340.00
HardSwish	48.96	166.80	140.33	308.80
ReLU	48.03	93.17	103.73	283.40
ActNAS8m.1	48.50	188.40	136.85	319.60
ActNAS8m.2	48.60	143.80	114.33	305.48

4. Results

We conducted our experiments on YOLO5n and YOLO8m [8] models and the selection is to cover two family of YOLO models with different levels of complexity. Initial

benchmarks were performed on YOLO5n model for Pascal VOC dataset [5] to avoid training multiple models on the larger COCO dataset [12]. Still, all the final benchmarking is performed on COCO dataset. Tables 1 and 2 summarize mAP, RAM (NPU only), and latency values for both the reference models and our mixed activation models for YOLO5n model. The baseline models consist of COCO-trained YOLO5 using SiLU, ReLU, and Hardswish activations, respectively. The LZCM1 model aims to minimize overall latency by mixing SiLU and ReLU activations using a simple search approach, while the LZCM2 model prioritizes maximizing accuracy by combining the more accurate activations, SiLU and Hardswish. Both models achieve mAP values slightly lower, but very close to SiLU and Hardswish reference models. In terms of latency, the LZCM1 model performs slightly better than the SiLU model across all devices (e.g., 21.87 ms vs. 22.35 ms on the NPU). The SiLU/Hardswish mixed activation model demonstrates a better accuracy-latency trade-off, being 12.5% faster than the SiLU model but 5.3% slower than the Hardswish model on the NPU. A similar trend

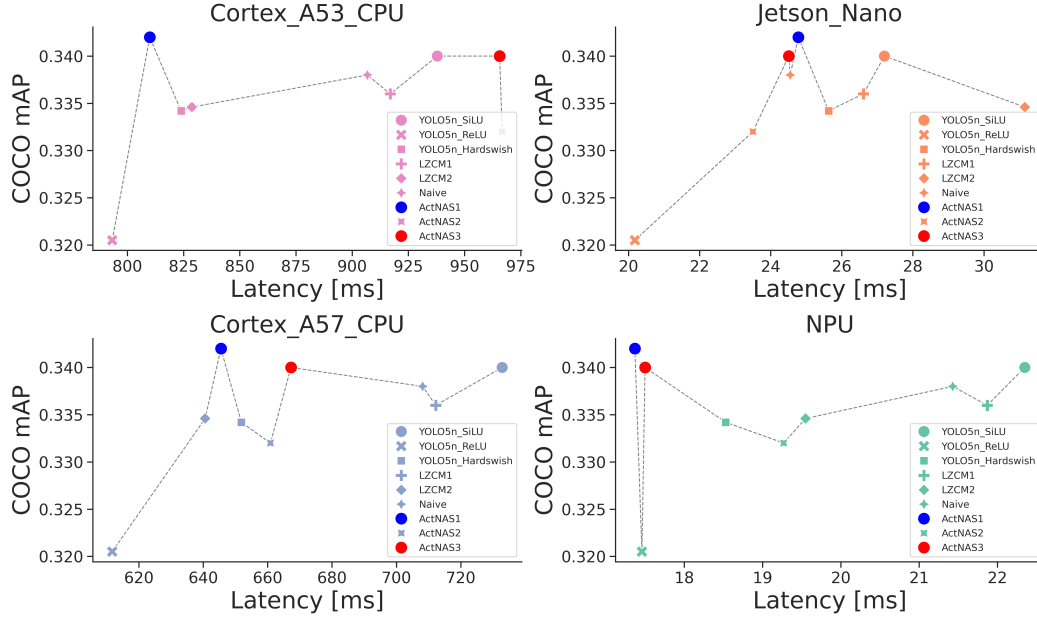


Figure 6. Performance of YOLO5n reference models and ActNAS models on different edge devices.

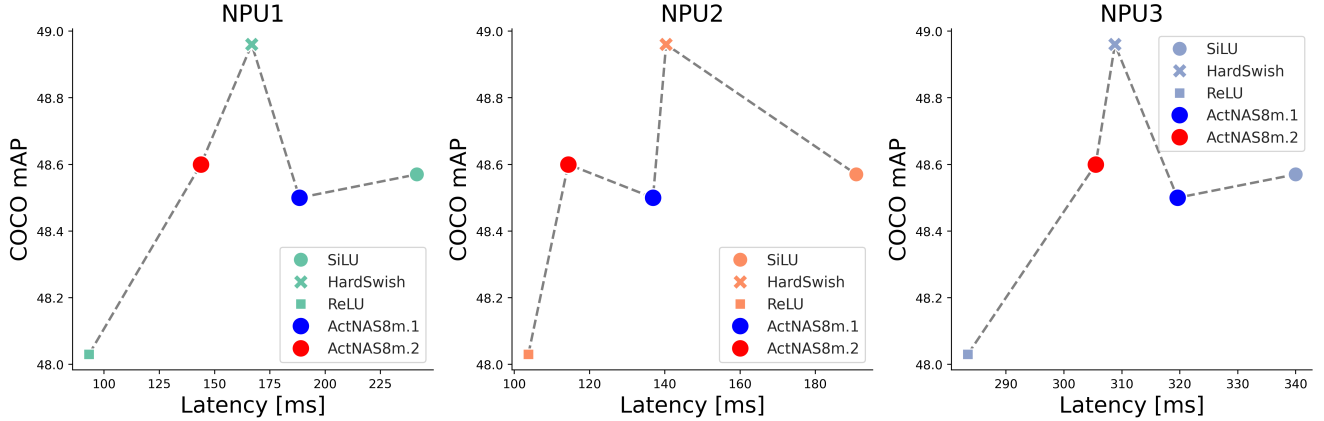


Figure 7. Performance of YOLO8m reference models and ActNAS models on different edge NPUs.

is observed on A53 and A57 CPUs, as well as the Jetson Nano GPU. Lastly, the naive approach of replacing the first three activations from SiLU to ReLU resulted in slightly slower performance compared to other baseline models, but the drop in mAP was less than 1% compared to the SiLU model.

ActNAS5n.1 and ActNAS5n.2 were generated using our activation NAS approach, targeting the NPU as the primary device, as discussed in the previous section. The mAP of ActNAS5n.1 model is within 1% drop compared to both SiLU and HardSwish variants but this model has lowest latency showing 6.26%, 3.32%, 0.97% and 1.69% improvements in efficiency on the NPU1, Jetson Nano GPU, Cortex A57 CPU and A53 CPU, respectively, compared to the

Hardswish model. It also outperformed the SiLU model by 22.28%, 8.90%, 11.92%, and 13.64% on the these devices. Furthermore, we found that ActNAS5n.1 required 58% less memory than the SiLU model on NPU1. ActNAS5n.2, generated with the Jetson Nano GPU as the target, outperformed both the baseline and HardSwish models on Jetson Nano, highlighting the hardware-aware nature of our search method, as it adapts to different hardware profiles. We also observed that the best latency-accuracy trade-off models tends to use ReLU variants (ReLU6, LeakyReLU, and ReLU) for initial layers. None of the ActNAS models utilized SiLU or Hardswish in the first two layers. This suggests that replacing the initial layers with more efficient activations and following them with high-performance ac-

tivations selected via NAS to minimize accuracy loss produces optimal models across a range of hardware devices. Figure 6 compares our models with baseline models, showing that Activation NAS models offer the best accuracy-latency trade-off on all tested hardware. In order to test the scalability of this approach, we repeated these experiments with YOLO8m models which has more activation layers compared to YOLO5n. We observed that the impact of different activations is more prominent on NPUs as latency is not only affected by the complexity of operations but also by the memory usage. Therefore, we focused all the YOLO8m experiments on three different NPUs. Table 3 shows the mAP/latency trade-off of ActNAS8m.1 and ActNAS8m.2 models compared to baseline models. These models are created to be NPU2 hardware aware and it is evident that ActNAS8m.2 model is 1.66x faster compared to the SiLU model without significant impact on accuracy of the model. Figure 7 shows the mAP-latency curve for ActNAS YOLO8m models compared to baseline model.

5. Hardware Awareness

As mentioned in section 3, all of the ActNAS models use one of the edge device for latency/memory computation. Since the latency and memory impact of each change in the model correlates to the hardware architecture, memory limitations and compiler optimization, we expect the resulting optimized model to have the best performance on that hardware. In other words, the model is "device" aware. For example, ActNAS5n.1, ActNAS5n.2, ActNAS8m.1 and ActNAS8m.2 are NPU1, Jetson Nano GPU, NPU2 and NPU2 aware models. The bar chart in Figure 8 shows the speedup of NPU1 aware ActNAS5n.1 and NPU2 aware ActNAS8m.2 speed up on five different devices. NPU1 and NPU2 have similar architecture but with different memory configuration. We can see that both NPU aware models provide highest speedup on these devices but very low speedup on other devices. These results also suggest that the NAS is able to learn model configuration suited best for these devices.

6. Conclusion

In this paper, we present an approach called Activation NAS (ActNAS), designed to generate efficient YOLO model architectures through a unique search space of varied activation functions for every layer. Our mixed-activation models deliver performance comparable to SiLU/Hardswish-based models on the COCO dataset, while offering up to 1.66x greater efficiency, depending on the hardware. This efficiency is achieved using a Neural Architecture Search (NAS) approach that leverages the zero-cost NWOT score, enabling us to skip the costly process of training a vast number of candidate models, thus making model optimization

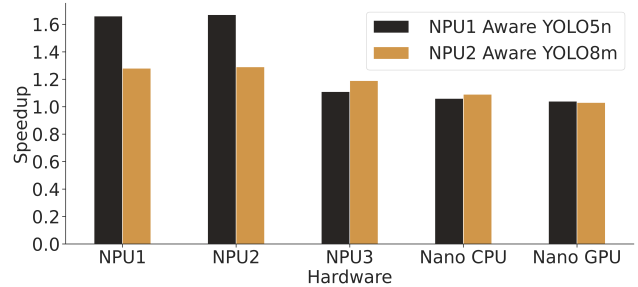


Figure 8. ActNAS model speedup on different hardware where the model is optimized for NPU1 (NPU1 and NPU2 have similar configuration)

tion more practical. Our experiments suggest that, rather than using a single activation function across all model layers, a combination of latency and accuracy constraints can guide the creation of custom mixed-activation models tailored to meet specific accuracy, latency, and memory requirements. Furthermore, we demonstrate that models optimized with particular hardware in mind (hardware-aware models) achieve peak performance on that hardware, yet may under-perform on others. This finding is crucial for model accelerators and NPUs with varying architectures, where hardware-agnostic optimizations often fail to yield efficient models.

References

- [1] Mohamed S Abdelfattah, Abhinav Mehrotra, Łukasz Dudziak, and Nicholas D Lane. Zero-cost proxies for lightweight nas. *arXiv preprint arXiv:2101.08134*, 2021. 2
- [2] AF Agarap. Deep learning using rectified linear units (relu). *arXiv preprint arXiv:1803.08375*, 2018. 1, 2
- [3] Yevgeniy Bolyanskiy and Serhii Kostyuk. Adaptive hybrid activation function for deep neural networks. *System research and information technologies*, pages 87–96, 2022. 2
- [4] Stefan Elfving, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks*, 107:3–11, 2018. 1, 2
- [5] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman. The PASCAL Visual Object Classes Challenge 2012 (VOC2012) Results, 2012. 6
- [6] Jocher Glenn, 2022. 2
- [7] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1314–1324, 2019. 1, 2
- [8] Glenn Jocher. Ultralytics yolov5, 2020. 5, 6
- [9] Ivan Lazarevich, Matteo Grimaldi, Ravish Kumar, Saptarshi Mitra, Shahrukh Khan, and Sudhakar Sah. Yolobench: Benchmarking efficient object detectors on embedded systems, 2023. 2

- [10] Chuyi Li, Lulu Li, Yifei Geng, Hongliang Jiang, Meng Cheng, Bo Zhang, Zaidan Ke, Xiaoming Xu, and Xiangxiang Chu. Yolov6 v3. 0: A full-scale reloading. *arXiv preprint arXiv:2301.05586*, 2023. 2
- [11] Guihong Li, Yuedong Yang, Kartikeya Bhardwaj, and Radu Marculescu. Zico: Zero-shot nas via inverse coefficient of variation on gradients. *arXiv preprint arXiv:2301.11300*, 2023. 2
- [12] Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, and Piotr Dollár. Microsoft coco: Common objects in context, 2014. cite arxiv:1405.0312. 1, 6
- [13] TensorFlow Lite. Tensorflow lite for mobile and edge, 2024. Accessed: 2024-07-16. 3
- [14] Andre Martins and Ramon Astudillo. From softmax to sparsemax: A sparse model of attention and multi-label classification. In *International conference on machine learning*, pages 1614–1623. PMLR, 2016. 2
- [15] Joe Mellor, Jack Turner, Amos Storkey, and Elliot J Crowley. Neural architecture search without training. In *International Conference on Machine Learning*, pages 7588–7598. PMLR, 2021. 2
- [16] Stuart Mitchell, Stuart Mitchell Consulting, Michael O’Sullivan, and Iain Dunning. Pulp: A linear programming toolkit for python, 2022. 5
- [17] Quynh N Nguyen and Marco Mondelli. Global convergence of deep networks with one wide layer followed by pyramidal topology. *Advances in Neural Information Processing Systems*, 33:11961–11972, 2020. 2
- [18] ONNX. Open neural network exchange, 2024. Accessed: 2024-07-16. 3
- [19] Tim Pearce, Alexandra Brintrup, and Jun Zhu. Understanding softmax confidence and uncertainty. *arXiv preprint arXiv:2106.04972*, 2021. 2
- [20] P Privietha and V Joesph Raj. Hybrid activation function in deep learning for gait analysis. In *2022 International Virtual Conference on Power Engineering Computing and Control: Developments in Electric Vehicles and Energy Sector for Sustainable Future (PECCON)*, pages 1–7. IEEE, 2022. 2
- [21] Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017. 2
- [22] Sudhakar Sah, Darshan C Ganji, Matteo Grimaldi, Ravish Kumar, Alexander Hoffman, Honnesh Rohmetra, and Ehsan Saboori. Mcubench: A benchmark of tiny object detectors on mcus. *arXiv preprint arXiv:2409.18866*, 2024. 3, 4
- [23] Juan Terven, Diana-Margarita Córdova-Esparza, and Julio-Alejandro Romero-González. A comprehensive review of yolo architectures in computer vision: From yolov1 to yolov8 and yolo-nas. *Machine Learning and Knowledge Extraction*, 5(4):1680–1716, 2023. 1, 2
- [24] CA Trauth Jr and RE Woolsey. Integer linear programming: a study in computational efficiency. *Management Science*, 15(9):481–493, 1969. 5
- [25] Ao Wang, Hui Chen, and Lihao Liu. Yolov10: Real-time end-to-end object detection. *arXiv preprint arXiv:2405.14458*, 2024. 1
- [26] Bing Xu. Empirical evaluation of rectified activations in convolutional network. *arXiv preprint arXiv:1505.00853*, 2015. 2
- [27] Zhenyu Zhu, Fanghui Liu, Grigorios Chrysos, and Volkan Cevher. Generalization properties of nas under activation and skip connection search. *Advances in Neural Information Processing Systems*, 35:23551–23565, 2022. 2
- [28] B Zoph. Neural architecture search with reinforcement learning. *arXiv preprint arXiv:1611.01578*, 2016. 2